

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Knowledge Representation Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I G.Karthik Reddy-192525047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Karthik Reddy

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

1. Smart Healthcare Diagnosis System

1. Propositional Logic

Let:

- F = Ravi has fever
- C = Ravi has cough
- B = Ravi has body pain
- FL = Ravi has flu
- VI = Ravi has viral infection
- R = Ravi should rest

Facts:

- $F = \text{True}$
- $C = \text{True}$
- $B = \text{True}$

Rules:

1. Fever and cough $\rightarrow$ Flu
 $F \wedge C \rightarrow FL$
2. Flu and body pain $\rightarrow$ Viral infection
 $FL \wedge B \rightarrow VI$
3. Viral infection $\rightarrow$ Rest
 $VI \rightarrow R$

2. First-Order Logic

Predicates:

- Fever(x)
- Cough(x)
- BodyPain(x)
- Flu(x)
- ViralInfection(x)
- Rest(x)

Facts:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Rules:

$\forall x [(Fever(x) \wedge Cough(x)) \rightarrow Flu(x)]$

$\forall x [(Flu(x) \wedge BodyPain(x)) \rightarrow ViralInfection(x)]$

$\forall x [ViralInfection(x) \rightarrow Rest(x)]$

3. Knowledge Base**Facts:**

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Rules:

$Fever(x) \wedge Cough(x) \rightarrow Flu(x)$

$Flu(x) \wedge BodyPain(x) \rightarrow ViralInfection(x)$

$ViralInfection(x) \rightarrow Rest(x)$

4. Unification**Example 1**

Fever(x)

Fever(Ravi)

Substitution:

$\{x/Ravi\}$

Therefore:

$Fever(x) = Fever(Ravi)$

Example 2

Flu(x)

Flu(Ravi)

Substitution:

$\{x/Ravi\}$

So the **MGU (Most General Unifier)** is:

$\{x/Ravi\}$

5. Forward Chaining

Start with:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

Step 1

Fever + Cough $\rightarrow$ Flu

Flu(Ravi)

Step 2

Flu + BodyPain $\rightarrow$ Viral Infection

ViralInfection(Ravi)

Step 3

Viral Infection $\rightarrow$ Rest

Rest(Ravi)

Final conclusion:

Ravi has a viral infection and should be advised to rest.

6. Backward Chaining

Goal:

Rest(Ravi)

Find a rule that produces Rest:

$ViralInfection(x) \rightarrow Rest(x)$

Therefore, we need:

$ViralInfection(Ravi)$

Find its rule:

$Flu(x) \wedge BodyPain(x) \rightarrow ViralInfection(x)$

We need:

Flu(Ravi)

BodyPain(Ravi)

BodyPain(Ravi) is already a fact.

To prove Flu(Ravi):

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

We have:

Fever(Ravi)

Cough(Ravi)

Therefore:

Flu(Ravi)

$\rightarrow \text{ViralInfection}(\text{Ravi})$

$\rightarrow \text{Rest}(\text{Ravi})$

Hence, Ravi needs rest.

7. Resolution

To prove:

Flu(Ravi)

Convert the rule:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

into CNF:

$\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$

Facts:

Fever(Ravi)

Cough(Ravi)

Negate the query:

$\neg \text{Flu}(\text{Ravi})$

Resolution gives:

Flu(Ravi)

which contradicts:

$\neg \text{Flu}(\text{Ravi})$

Therefore, the query is proved.

8. Final Inference

Fever(Ravi)

+

Cough(Ravi)

↓

Flu(Ravi)

+

BodyPain(Ravi)

↓

ViralInfection(Ravi)

↓

Rest(Ravi)

Final answer: Ravi is inferred to have a **viral infection**, and the system recommends **rest**. The source specifically asks these reasoning steps for the healthcare case.

2. Intelligent Loan Approval System

The assessment describes Arun as having a stable job, high credit score, and sufficient income, with rules connecting employment, income, repayment history, risk, and approval.

1. Entities, Facts, Predicates and Rules

Entity

Arun

Predicates

StableJob(x)

HighCreditScore(x)

SufficientIncome(x)

GoodRepayment(x)

ExistingLoan(x)

PreliminaryApproval(x)

LowRisk(x)

LoanApproved(x)

Facts

StableJob(Arun)

HighCreditScore(Arun)

SufficientIncome(Arun)

GoodRepayment(Arun)

NoExistingLoan(Arun)

2. FOL Representation

StableJob(Arun)

HighCreditScore(Arun)

SufficientIncome(Arun)

GoodRepayment(Arun)

Rules:

$\forall x [(StableJob(x) \wedge SufficientIncome(x))$
 $\rightarrow PreliminaryApproval(x)]$

$\forall x [(HighCreditScore(x) \wedge GoodRepayment(x))$
 $\rightarrow LowRisk(x)]$

$\forall x [(LowRisk(x) \wedge PreliminaryApproval(x))$
 $\rightarrow LoanApproved(x)]$

3. Knowledge Base — 10 Facts

1. StableJob(Arun)

2. HighCreditScore(Arun)

3. SufficientIncome(Arun)

4. GoodRepayment(Arun)

5. NoExistingLoan(Arun)

6. AgeAbove21(Arun)

7. ValidDocuments(Arun)

8. PermanentAddress(Arun)

9. RegularSalary(Arun)

10. BankAccount(Arun)

8 Rules

R1: StableJob(x) $\wedge$ SufficientIncome(x)

→ PreliminaryApproval(x)

R2: HighCreditScore(x) $\wedge$ GoodRepayment(x)

→ LowRisk(x)

R3: LowRisk(x) $\wedge$ PreliminaryApproval(x)

→ LoanApproved(x)

R4: RegularSalary(x) $\wedge$ StableJob(x)

→ IncomeStable(x)

R5: ValidDocuments(x) $\wedge$ BankAccount(x)

→ VerifiedCustomer(x)

R6: NoExistingLoan(x) $\wedge$ GoodRepayment(x)

→ GoodFinancialHistory(x)

R7: IncomeStable(x) $\wedge$ GoodFinancialHistory(x)

→ FinanciallyEligible(x)

R8: FinanciallyEligible(x) $\wedge$ VerifiedCustomer(x)

→ FinalApproval(x)

4. Unification

Given:

StableJob(x)

StableJob(Arun)

Substitution:

{x/Arun}

Therefore:

StableJob(x) = StableJob(Arun)

Another example:

HighCreditScore(x)

HighCreditScore(Arun)

MGU:

{x/Arun}

5. Forward Chaining

Initial facts:

StableJob(Arun)

SufficientIncome(Arun)

HighCreditScore(Arun)

GoodRepayment(Arun)

Apply R1:

PreliminaryApproval(Arun)

Apply R2:

LowRisk(Arun)

Apply R3:

LoanApproved(Arun)

Conclusion:

Arun can be recommended for loan approval.

6. Backward Chaining

Goal:

LoanApproved(Arun)

Rule:

LowRisk(x) $\wedge$ PreliminaryApproval(x)

$\rightarrow$ LoanApproved(x)

Need:

LowRisk(Arun)

PreliminaryApproval(Arun)

Both can be proved from the given facts.

Therefore:

LoanApproved(Arun)

is proved.

7. Resolution

Rule:

$\text{LowRisk}(x) \wedge \text{PreliminaryApproval}(x)$

$\rightarrow \text{LoanApproved}(x)$

CNF:

$\neg \text{LowRisk}(x) \vee \neg \text{PreliminaryApproval}(x)$

$\vee \text{LoanApproved}(x)$

Given:

$\text{LowRisk}(\text{Arun})$

$\text{PreliminaryApproval}(\text{Arun})$

Negate query:

$\neg \text{LoanApproved}(\text{Arun})$

Resolution produces a contradiction.

Therefore:

LoanApproved(Arun) is true.

8. Knowledge Engineering

Knowledge engineering helps by:

- Clearly defining entities and predicates.
- Separating facts from rules.
- Making rules easy to modify.
- Supporting automated reasoning.
- Reducing inconsistent decision-making.
- Making the approval process easier to explain.

3. Intelligent Vehicle Fault Diagnosis

The assessment gives rules for dead battery, fuel problems, overheating, and immediate service, with Car101 having a starting problem, dim headlights, and high temperature.

1. Knowledge Representation

Facts

DoesNotStart(Car101)

DimHeadlights(Car101)

HighTemperature(Car101)

Predicates

DoesNotStart(x)

DimHeadlights(x)

DeadBattery(x)

EngineCrank(x)

FuelProblem(x)

HighTemperature(x)

LowCoolant(x)

Overheating(x)

NeedsService(x)

2. Five FOL Rules

R1: DoesNotStart(x) $\wedge$ DimHeadlights(x)

→ DeadBattery(x)

R2: EngineCrank(x) $\wedge$ DoesNotStart(x)

→ FuelProblem(x)

R3: HighTemperature(x) $\wedge$ LowCoolant(x)

→ Overheating(x)

R4: Overheating(x)

→ NeedsService(x)

R5: DeadBattery(x)

→ NeedsService(x)

3. Propositional Logic

Let:

- D = Car101 does not start
- H = Car101 has dim headlights
- B = Car101 has dead battery
- T = Car101 has high temperature
- O = Car101 has overheating
- S = Car101 needs service

Rules:

$$D \wedge H \rightarrow B$$

$$T \wedge \text{LowCoolant} \rightarrow O$$

$$O \rightarrow S$$

$$B \rightarrow S$$

4. Unification

DoesNotStart(x)

DoesNotStart(Car101)

Substitution:

$\{x/\text{Car101}\}$

Therefore:

DoesNotStart(Car101)

Similarly:

DimHeadlights(x)

DimHeadlights(Car101)

gives:

$\{x/\text{Car101}\}$

5. Forward Chaining

Given:

DoesNotStart(Car101)

DimHeadlights(Car101)

HighTemperature(Car101)

Apply R1:

DeadBattery(Car101)

Apply R5:

NeedsService(Car101)

Conclusion:

Car101 has a possible dead-battery problem and needs service.

The high-temperature fact alone does **not** prove overheating because the supplied rule also requires low coolant. This follows directly from the rule given in the assessment.

6. Backward Chaining

Goal:

NeedsService(Car101)

Possible rule:

$\text{DeadBattery}(x) \rightarrow \text{NeedsService}(x)$

Need:

DeadBattery(Car101)

Rule:

$\text{DoesNotStart}(x) \wedge \text{DimHeadlights}(x)$

$\rightarrow \text{DeadBattery}(x)$

We have both facts.

Therefore:

DeadBattery(Car101)

$\rightarrow \text{NeedsService}(\text{Car101})$

Query is proved.

7. Resolution

Given:

DoesNotStart(Car101)

DimHeadlights(Car101)

Rule:

$\neg \text{DoesNotStart}(x) \vee \neg \text{DimHeadlights}(x)$

$\vee \text{DeadBattery}(x)$

Negate the query:

$\neg \text{DeadBattery}(\text{Car101})$

Resolution with the two facts derives:

$\text{DeadBattery}(\text{Car101})$

This contradicts the negated query.

Therefore, **Car101 has a dead-battery fault.**

8. Final Diagnosis

DoesNotStart + DimHeadlights

↓

Dead Battery

↓

Needs Service

Final answer: Car101 likely has a dead-battery problem and should be sent for service.

4. Intelligent Student Academic Advising System

The assessment specifies attendance below 75%, two or more failed courses, high attendance with good performance, and prerequisite completion as the main rules. Priya has 68% attendance, two failed courses, and has completed a prerequisite.

1. Knowledge Engineering Requirements

The system needs:

- **Entities:** Student, Course
- **Facts:** Attendance, failed courses, prerequisites
- **Predicates:** LowAttendance, FailedCourses, RequiresCounseling, etc.
- **Rules:** Academic advising rules
- **Inference engine:** Forward and backward chaining
- **Knowledge base:** Facts + rules

- **Output:** Recommendations

2. FOL Rules

$\forall x [AttendanceBelow75(x) \rightarrow AttendanceCounseling(x)]$

$\forall x [FailedTwoOrMore(x) \rightarrow AcademicCounseling(x)]$

$\forall x [(HighAttendance(x) \wedge GoodPerformance(x))$
 $\rightarrow AdvancedCourseRecommendation(x)]$

$\forall x [(CompletedPrerequisite(x, c))$
 $\rightarrow CanRegister(x, c)]$

3. Knowledge Base

Facts

Attendance(Priya, 68)

FailedCourses(Priya, 2)

CompletedPrerequisite(Priya, AdvancedAI)

Derived facts

AttendanceBelow75(Priya)

FailedTwoOrMore(Priya)

4. Unification

Example:

CompletedPrerequisite(x, AdvancedAI)

CompletedPrerequisite(Priya, AdvancedAI)

Substitution:

$\{x/Priya\}$

Another example:

CanRegister(x, AdvancedAI)

CanRegister(Priya, AdvancedAI)

Substitution:

$\{x/Priya\}$

5. Forward Chaining

Initial facts:

Attendance(Priya, 68)

FailedCourses(Priya, 2)

CompletedPrerequisite(Priya, AdvancedAI)

Since:

$68 < 75$

derive:

AttendanceCounseling(Priya)

Since Priya failed two courses:

AcademicCounseling(Priya)

Since the prerequisite is completed:

CanRegister(Priya, AdvancedAI)

Recommendations

1. Attendance counseling
2. Academic counseling
3. Eligible to register for Advanced AI

6. Backward Chaining

Query:

AcademicCounseling(Priya)

Rule:

FailedTwoOrMore(x)

→ AcademicCounseling(x)

Need:

FailedTwoOrMore(Priya)

This is supported by:

FailedCourses(Priya, 2)

Therefore:

AcademicCounseling(Priya)

is proved.

7. Resolution

Rule:

$\text{FailedTwoOrMore}(x) \rightarrow \text{AcademicCounseling}(x)$

CNF:

$\neg \text{FailedTwoOrMore}(x) \vee \text{AcademicCounseling}(x)$

Given:

$\text{FailedTwoOrMore}(\text{Priya})$

Negated query:

$\neg \text{AcademicCounseling}(\text{Priya})$

Resolution produces a contradiction.

Therefore:

Priya requires academic counseling.

8. Advantages and Limitations

Advantages

- Easy to understand.
- Rules are transparent.
- Easy to explain recommendations.
- Gives consistent decisions.
- Easy to update rules.

Limitations

- Cannot easily handle uncertain information.
- Large rule bases can become complex.
- Requires accurate facts.
- Exceptions may require additional rules.

5. Cybersecurity Threat Detection System

The assessment describes failed logins, unusual login locations, unauthorized file access, possible security breaches, and high-priority alerts for User101.

1. Knowledge Representation Model

Entity

User101

Predicates

FailedLogin(x)

UnusualLocation(x)

SuspiciousActivity(x)

UnauthorizedAccess(x)

SecurityBreach(x)

HighPriorityAlert(x)

BruteForceAttack(x)

2. Propositional Logic

Let:

- F = Multiple failed logins
- U = Unusual location
- S = Suspicious activity
- A = Unauthorized file access
- B = Security breach
- H = High-priority alert

Rules:

$F \rightarrow \text{BruteForceAttack}$

$F \wedge U \rightarrow S$

$S \wedge A \rightarrow B$

$B \rightarrow H$

3. FOL Representation

$\forall x [\text{MultipleFailedLogins}(x)$

$\rightarrow \text{BruteForceAttack}(x)]$

$\forall x [(\text{MultipleFailedLogins}(x) \wedge \text{UnusualLocation}(x))$

$\rightarrow \text{SuspiciousActivity}(x)]$

$\forall x [(\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedAccess}(x))$

$\rightarrow \text{SecurityBreach}(x)]$

$\forall x [\text{SecurityBreach}(x)$

$\rightarrow \text{HighPriorityAlert}(x)]$

4. Knowledge Base — 10 Facts

1. $\text{MultipleFailedLogins}(\text{User101})$

2. $\text{UnusualLocation}(\text{User101})$

3. $\text{UnauthorizedAccess}(\text{User101})$

4. $\text{LoginAttempt}(\text{User101})$

5. $\text{NetworkActivity}(\text{User101})$

6. $\text{UserAccountActive}(\text{User101})$

7. $\text{ConfidentialFile}(\text{User101})$

8. $\text{AuthenticationFailure}(\text{User101})$

9. $\text{SuspiciousLogin}(\text{User101})$

10. $\text{FileAccessAttempt}(\text{User101})$

8 Rules

R1: $\text{MultipleFailedLogins}(x)$

$\rightarrow \text{BruteForceAttack}(x)$

R2: $\text{MultipleFailedLogins}(x) \wedge \text{UnusualLocation}(x)$

$\rightarrow \text{SuspiciousActivity}(x)$

R3: $\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedAccess}(x)$

$\rightarrow \text{SecurityBreach}(x)$

R4: $\text{SecurityBreach}(x)$

$\rightarrow \text{HighPriorityAlert}(x)$

R5: AuthenticationFailure(x)

→ SuspiciousLogin(x)

R6: SuspiciousLogin(x) $\wedge$ UnusualLocation(x)

→ SuspiciousActivity(x)

R7: ConfidentialFile(x) $\wedge$ UnauthorizedAccess(x)

→ SensitiveFileAccess(x)

R8: SensitiveFileAccess(x) $\wedge$ SuspiciousActivity(x)

→ SecurityBreach(x)

5. Unification and Substitution

Given:

MultipleFailedLogins(x)

MultipleFailedLogins(User101)

MGU:

{x/User101}

Therefore:

MultipleFailedLogins(User101)

Another example:

UnauthorizedAccess(x)

UnauthorizedAccess(User101)

MGU:

{x/User101}

6. Forward Chaining

Initial facts:

MultipleFailedLogins(User101)

UnusualLocation(User101)

UnauthorizedAccess(User101)

Step 1

Multiple failed logins:

BruteForceAttack(User101)

Step 2

Failed logins + unusual location:

SuspiciousActivity(User101)

Step 3

Suspicious activity + unauthorized access:

SecurityBreach(User101)

Step 4

Security breach:

HighPriorityAlert(User101)

Final conclusion:

A high-priority security alert should be generated for User101.

7. Backward Chaining

Query:

HighPriorityAlert(User101)

Rule:

$\text{SecurityBreach}(x) \rightarrow \text{HighPriorityAlert}(x)$

Need:

SecurityBreach(User101)

Rule:

$\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedAccess}(x)$

$\rightarrow \text{SecurityBreach}(x)$

We have:

UnauthorizedAccess(User101)

and can derive:

SuspiciousActivity(User101)

from:

MultipleFailedLogins(User101)

+

UnusualLocation(User101)

Therefore:

SecurityBreach(User101)

→ HighPriorityAlert(User101)

Query is proved.

8. Resolution

Rule:

SuspiciousActivity(x) $\wedge$ UnauthorizedAccess(x)

→ SecurityBreach(x)

CNF:

$\neg$ SuspiciousActivity(x)

$\vee \neg$ UnauthorizedAccess(x)

$\vee$ SecurityBreach(x)

Given:

SuspiciousActivity(User101)

UnauthorizedAccess(User101)

Negated query:

$\neg$ SecurityBreach(User101)

Resolution derives:

SecurityBreach(User101)

which contradicts the negated query.

Therefore:

A possible security breach exists.

9. Forward vs Backward Chaining

Feature	Forward Chaining	Backward Chaining
Direction	Facts → Conclusion	Goal → Facts
Best for	Continuous monitoring	Specific investigation
Cybersecurity use	Detects new threats automatically	Investigates a specific alert
Starting point	Known events	Query
Example	Failed login → alert	“Is User101 under attack?”

